UJIAN TENGAH SEMESTER

NAMA : TRISNO TRIA NURAHMAN

NIM : 20230801328

SISTEM BASIS DATA TERDISTRIBUSI

BAGIAN A- ESSAY

A1. Fragmentasi Horizontal dan Vertikal pada Relasi Pegawai (NIP, Nama, Jabatan, Gaji, Unit)

Konsep :

- Fragmentasi horizontal membagi relasi menurut tuple (baris) dengan operasi selection: $R_i = \sigma_P_i(R)$. Setiap fragmen memiliki skema yang sama dengan relasi asal tetapi hanya berisi sebagian baris.
- Fragmentasi vertikal membagi relasi menurut atribut (kolom) dengan operasi projection: $R_i = \pi_A_i(R)$. Setiap fragmen harus memuat primary key (NIP) agar dapat disambung kembali.
- Keduanya wajib memenuhi tiga aturan kebenaran: completeness (tidak ada data hilang), reconstruction (relasi asal dapat dibentuk ulang), dan *disjointness* (tidak tumpang tindih, kecuali atribut kunci pada fragmen vertikal).

a) Contoh pemecahan relasi

Horizontal berdasarkan Unit:

P1 = σ Unit='Keuangan' (Pegawai)

P2 = σ Unit='SDM' (Pegawai)

P3 = σ Unit≠'Keuangan' ∧ Unit≠'SDM' (Pegawai)

Vertikal berdasarkan jenis data:

V1 = π NIP, Nama, Jabatan, Unit (Pegawai) → data umum kepegawaian

V2 = π NIP, Gaji (Pegawai) → data sensitif penggajian

b) Rekonstruksi relasi asal

- Horizontal: Pegawai = P1 ∪ P2 ∪ P3 (union seluruh fragmen).

- Vertikal: Pegawai = V1 ⋈ V2 dengan kondisi V1.NIP = V2.NIP (natural join pada kunci). Syaratnya NIP unik dan terdapat pada kedua fragmen.

c) Menentukan alokasi fragmen pada beberapa simpul

Alokasi adalah penentuan fragmen mana ditempatkan di simpul mana (*allocation problem*). Faktor yang dipertimbangkan:

1. Pola akses: query apa, seberapa sering, dan dari simpul mana. Fragmen ditempatkan di simpul yang paling sering mengaksesnya (lokalitas data).
2. Biaya komunikasi dan kapasitas jaringan antarsimpul.
3. Rasio baca terhadap tulis: fragmen yang banyak dibaca dan jarang diubah cocok direplikasi; fragmen yang sering diubah sebaiknya satu salinan.
4. Kapasitas penyimpanan dan pemerataan beban antarsimpul.
5. Keamanan dan otonomi, misalnya fragmen Gaji hanya di simpul yang berwenang.
6. Kebutuhan ketersediaan, yang menentukan perlu tidaknya salinan ganda.

Caranya: susun matriks pola akses (query × fragmen × simpul), hitung manfaat tiap kandidat penempatan, lalu pilih penempatan bermanfaat terbesar. Hasilnya dapat non-redundan (satu salinan) atau redundan (bereplikasi). Contoh: P1 di simpul Keuangan, P2 di simpul SDM, dan V2 (Gaji) di simpul payroll dengan akses terbatas

A2. Replikasi Sinkron vs Asinkron untuk Layanan Transaksi yang Harus Tetap Tersedia

Aspek	**Replikasi sinkron**	**Replikasi asinkron**
Cara kerja	Commit baru sukses setelah replika memberi konfirmasi (2PC atau kuorum).	Commit di primer lebih dulu; propagasi ke replika menyusul.
Konsistensi data	Kuat: semua salinan sama, tidak ada data hilang (RPO = 0).	Akhirnya konsisten (*eventual*): ada *replication lag*, replika bisa usang, RPO > 0.
Ketersediaan	Rendah saat gangguan: jika replika/pusat data lain mati, penulisan tertahan atau ditolak.	Tinggi: primer tetap menerima transaksi meskipun replika tidak terjangkau.

Aspek	Replikasi sinkron	Replikasi asinkron
Latensi	Lebih tinggi (menunggu round-trip antar pusat data).	Rendah.
Risiko utama	Layanan berhenti dan throughput turun.	Transaksi terakhir bisa hilang saat failover; konflik pada multi-master.

Rekomendasi: replikasi asinkron (primary-backup) dengan failover otomatis.
Justifikasi:

- Persyaratan utama dalam kasus ini adalah bahwa layanan harus tetap tersedia ketika ada gangguan pusat data. Dalam replikasi sinkron, kegagalan satu pusat data menghentikan penulisan di pusat data yang sehat, sehingga persyaratan tidak terpenuhi.
- Latensi rendah menjaga performa transaksi.
- Kekurangan asinkron (salinan yang tertinggal) diminimalkan melalui pengiriman jurnal transaksi (WAL) yang berkelanjutan, pengawasan keterlambatan replikasi, ID transaksi yang bersifat idempoten, serta rekonsiliasi transaksi "ekor" setelah terjadinya failover.
- Jika peraturan menetapkan bahwa tidak boleh ada kehilangan data apa pun, solusinya adalah dengan menggunakan replikasi sinkron yang berbasis kuorum di tiga tempat (dua tempat untuk data dan satu untuk saksi), sehingga mayoritas dapat tetap melakukan penulisan meskipun satu lokasi tidak berfungsi.

A3. Join Pesanan (Simpul A) dan Pelanggan (Simpul B) pada id_pelanggan, Kueri dari Simpul A

Simbol dan asumsi

- n_P, s_P : jumlah tuple dan ukuran tuple Pesanan (byte)
- n_L, s_L : jumlah tuple dan ukuran tuple Pelanggan (byte)
- s_k : ukuran atribut id_pelanggan (byte)
- d : jumlah nilai id_pelanggan yang berbeda di Pesanan ($d \leq n_P$)
- σ : proporsi tuple Pelanggan yang cocok dengan Pesanan ($0 \leq \sigma \leq 1$)
- Biaya transmisi = jumlah byte yang dikirim. Biaya pemrosesan lokal dan overhead tetap per pesan diabaikan.

a) **Strategi 1: kirim seluruh relasi Pelanggan ke simpul A**

$C_1 = n_L \times s_L$

b) **Strategi 2: semi-join**

1. A mengirim π id_pelanggan (Pesanan) ke B. Biayanya $d \times s_k$.
2. B menghitung Pelanggan ⋉ Pesanan dan mengirim hanya tuple yang cocok ke A. Biayanya $\sigma \times n_L \times s_L$.
3. A melakukan join akhir secara lokal.

$C_2 = d \cdot s_k + \sigma \cdot n_L \cdot s_L$

c) **Kondisi semi-join lebih hemat**

$C_2 < C_1 \iff d \cdot s_k + \sigma \cdot n_L \cdot s_L < n_L \cdot s_L$

$\iff d \cdot s_k < (1 - \sigma) \cdot n_L \cdot s_L$

$\iff \sigma < 1 - (d \cdot s_k) / (n_L \cdot s_L)$

Semi-join bermanfaat saat hanya sejumlah kecil Pelanggan yang sesuai (σ kecil), jumlah kunci yang dikirim (d) rendah, dan ukuran kunci jauh lebih kecil daripada ukuran tuple Pelanggan ($s_k \ll s_L$). Di sisi lain, jika hampir semua pelanggan memenuhi syarat ($\sigma \approx 1$), semi-join malah menjadi lebih mahal karena ada biaya tambahan untuk mengirim kunci.

Bila ada integritas referensial (setiap id_pelanggan di Pesanan ada di Pelanggan), maka $\sigma \cdot n_L = d$ sehingga syaratnya menjadi $d \cdot (s_k + s_L) < n_L \cdot s_L$.

d) **Strategi yang dipilih**

Dipilih semi-join. Pada data transaksi biasanya hanya sebagian pelanggan yang memiliki pesanan, id_pelanggan berukuran kecil (4–8 byte), dan tuple pelanggan jauh lebih besar (nama, alamat, kontak, dll.), sehingga syarat pada (c) terpenuhi.

Contoh angka: $n_L = 100.000$; $s_L = 200$ B; $d = 10.000$; $s_k = 8$ B; $\sigma = 0{,}1$.

- $C_1 = 100.000 \times 200 = 20.000.000$ B (± 20 MB)
- $C_2 = 10.000 \times 8 + 0{,}1 \times 100.000 \times 200 = 80.000 + 2.000.000 = 2.080.000$ B (± 2,08 MB)

Penghematan sekitar 90%. Jika pada data nyata σ mendekati 1, pilih Strategi 1.

A4. Protokol Two-Phase Commit untuk Transfer Dana antara Dua Rekening di Dua Simpul

Contoh: rekening X berada di simpul koordinator (K), rekening Y di simpul partisipan (P).
Transfer sebesar N dari X ke Y.

Fase 1: Prepare / voting

1. Koordinator memulai transaksi T dan menjalankan bagian lokalnya (debit X, kunci dipegang).
2. Koordinator *force-write* <PREPARE T> ke log, lalu mengirim pesan PREPARE ke partisipan.
3. Partisipan memeriksa apakah dapat melakukan bagiannya (rekening Y valid, kunci tersedia, constraint terpenuhi). Jika bisa: *force-write* <READY T> (beserta informasi undo/redo), lalu kirim VOTE-COMMIT (YES); sejak saat ini partisipan menahan kunci dan tidak boleh membatalkan sendiri. Jika tidak bisa: tulis <ABORT T>, batalkan lokal, kirim VOTE-ABORT (NO).

Fase 2: Commit / abort (keputusan global)

1. Koordinator mengumpulkan semua suara. Jika semua YES (termasuk suara lokalnya): *force-write* <COMMIT T>, lalu kirim GLOBAL-COMMIT. Jika ada NO atau timeout: *force-write* <ABORT T>, lalu kirim GLOBAL-ABORT.
2. Partisipan menerima keputusan, menulis <COMMIT T> atau <ABORT T> ke log, menerapkan atau membatalkan perubahan, melepas kunci, dan mengirim ACK.
3. Setelah semua ACK diterima, koordinator menulis <END T>.

Peran

- Koordinator: memulai protokol, mengumpulkan suara, mengambil dan mencatat keputusan global secara durable, mengirimkannya, menangani timeout, dan menyimpan keputusan untuk pemulihan.
- Partisipan: menjalankan bagian lokal, memberi suara, lalu patuh pada keputusan koordinator.

Jika partisipan gagal setelah vote YES dan sebelum menerima keputusan akhir

- Karena <READY T> sudah tertulis di log, atomicity tetap terjaga: perubahan T tidak hilang.
- Koordinator tetap berpegang pada keputusannya (yang sudah tercatat) dan mengirim ulang keputusan sampai ACK diterima. Keputusan tidak berubah hanya karena partisipan mati.

- Saat pulih, partisipan membaca log dan menemukan T berstatus READY tanpa COMMIT/ABORT, yaitu *in-doubt*. Ia tidak boleh memutuskan sendiri; ia harus menanyakan hasil ke koordinator (*termination/recovery protocol*): bila COMMIT maka dilakukan *redo*; bila ABORT atau koordinator tidak punya catatan (*presumed abort*) maka dilakukan *undo*.
- Selama menunggu, kunci pada data terkait tetap ditahan (*blocking*).

Jika partisipan mati *sebelum* memberi suara, koordinator mengalami timeout dan memutuskan ABORT.

A5. Dua Simpul Menyimpan Salinan Saldo yang Sama, Jaringan Terputus saat Transaksi Berjalan

a) **Kemungkinan inkonsistensi**

- Atomicity pecah: bila transaksi menulis ke kedua salinan dan jaringan putus di tengah commit tanpa protokol atomic commit, satu simpul bisa sudah commit sedangkan simpul lain belum (*partial commit*). Dengan 2PC, kondisi yang muncul adalah *in-doubt* dan terblokir.
- Replika menyimpang (split-brain): kedua partisi menerima update sendiri-sendiri. Contoh: saldo Rp1.000.000; simpul 1 menyetujui penarikan Rp600.000 dan simpul 2 menyetujui penarikan Rp700.000. Keduanya valid secara lokal, tetapi gabungannya membuat saldo negatif (lost update, double spending).
- Pembacaan usang (*stale read*): simpul yang terisolasi menampilkan saldo lama.

b) **Mekanisme deteksi atau pencegahan**

Pencegahan

- Atomic commit (2PC), yang lebih memilih menahan transaksi daripada membiarkan replika berbeda.
- Protokol kuorum ($W + R > N$ dan $W > N/2$) atau konsensus (Paxos/Raft): hanya partisi yang memegang mayoritas boleh menulis, sedangkan partisi minoritas menolak atau hanya membaca. Dengan hanya dua simpul diperlukan simpul ketiga (witness) agar mayoritas dapat terbentuk.
- Primary copy dengan pemilihan leader yang membutuhkan mayoritas, ditambah fencing token.

- Penguncian (*locking*) untuk mencegah penulisan konkuren.

Deteksi

- Heartbeat dan timeout untuk mendeteksi partisi.
- Nomor versi / version vector / timestamp untuk mengenali update yang konflik setelah jaringan pulih.
- Perbandingan log dan checksum antar-replika (*anti-entropy*).

Pemulihan setelah partisi pulih
Rekonsiliasi (misalnya menerapkan operasi sebagai selisih debit/kredit, bukan menimpa nilai) dan transaksi kompensasi.

c) **Tingkat konsistensi yang dapat dicapai dan trade-off ketersediaan**

Menurut teorema CAP, saat terjadi partisi sistem harus memilih antara konsistensi dan ketersediaan.

- Memilih konsistensi kuat (CP) (*one-copy serializability*, linearizability): partisi tanpa mayoritas menolak atau menunda transaksi. Data selalu benar, tetapi ketersediaan turun karena sebagian layanan tidak bisa menulis.
- Memilih ketersediaan (AP): kedua sisi tetap melayani, tetapi hanya mencapai konsistensi akhirnya (*eventual*) dan harus ada rekonsiliasi.

Untuk data saldo yang bersifat finansial, pilihan yang tepat adalah konsistensi kuat. Ketersediaan yang dikorbankan dapat diperkecil dengan mengizinkan pembacaan saldo yang ditandai “mungkin tidak terbaru” di partisi minoritas.

BAGIAN B – STUDI KASUS

B1. (Sub-CPMK-1.1) Fragmentasi Tabel Warga

Perbedaan fragmentasi horizontal dan vertikal

- Horizontal: memecah *baris* warga berdasarkan predikat (di sini kecamatan). Skema sama, isi berbeda.

- Vertikal: memecah *kolom* warga. Skema berbeda, tetapi setiap fragmen membawa NIK sebagai kunci penghubung.

Skema horizontal berdasarkan kecamatan

KA, KB, KC adalah himpunan kecamatan yang dilayani simpul A, B, C. Setiap kecamatan masuk tepat ke satu himpunan agar fragmen *disjoint* dan *complete*.

- W_A = σ kecamatan∈KA (Warga)
- W_B = σ kecamatan∈KB (Warga)
- W_C = σ kecamatan∈KC (Warga)

Skema vertikal (atribut identitas vs atribut layanan)

- W_ID = π NIK, nama, tanggal_lahir, alamat, kecamatan (Warga) → atribut identitas
- W_LY = π NIK, status_bantuan, nomor_telepon (Warga) → atribut layanan

Alokasi fragmen

Fragmen	Simpul	Alasan
W_A / W_B / W_C	A / B / C	Setiap kantor memproses warganya sendiri sesuai usulan pimpinan; akses lokal dan cepat meskipun jaringan antarkantor terputus.
W_ID	A (+ salinan di B)	Data stabil dan sering dibaca; selaras dengan usulan replikasi A–B untuk ketersediaan.
W_LY	C	Sering diperbarui petugas dan lebih sensitif (status bantuan); sekaligus menyeimbangkan beban karena A dan B sudah menampung Pembayaran dan Status_Permohonan.

Catatan: fragmentasi vertikal mengharuskan join lintas simpul untuk melihat data lengkap seorang warga, sehingga skema horizontal lebih cocok sebagai pembagian utama.

Rekonstruksi tabel Warga asli

- Dari fragmen horizontal: Warga = W_A ∪ W_B ∪ W_C.

- Dari fragmen vertikal: Warga = W_ID ⋈ W_LY dengan kondisi W_ID.NIK = W_LY.NIK.

B2. (Sub-CPMK-1.2) Replikasi Sinkron vs Asinkron untuk Warga dan Layanan_Surat

Sisi ketersediaan saat koneksi terputus

- Sinkron: setiap penulisan menunggu simpul lain. Saat koneksi putus di jam sibuk, penulisan (misalnya input permohonan surat) tertahan atau gagal, sehingga melanggar kebutuhan layanan tersedia setiap hari kerja.
- Asinkron: kantor tetap menulis ke salinan lokalnya dan perubahan dikirim saat jaringan pulih. Layanan tidak berhenti.

Sisi konsistensi data yang dibaca warga

- Sinkron: semua simpul selalu menampilkan data identik.
- Asinkron: ada jeda. Simpul lain dapat menampilkan data lama untuk sementara (nomor telepon lama, status permohonan belum terbaru). Bila satu record diubah di dua kantor saat terputus, dapat terjadi konflik.

Rekomendasi per tabel

Tabel	**Jenis replikasi**	**Justifikasi sesuai kasus**
Warga	Asinkron (primary-copy: tiap kecamatan punya satu simpul pemilik yang menerima update; salinan di A dan B bersifat baca)	Data induk yang banyak dibaca dan jarang berubah; keterlambatan beberapa detik atau menit tidak merugikan; layanan harus tetap berjalan saat jaringan terputus. Untuk keputusan kritis (mis. penyaluran bantuan) cek ulang ke simpul pemilik.
Layanan_Surat	Asinkron; dikelola per kecamatan oleh kantor asalnya, dengan salinan asinkron ke kantor pusat untuk laporan	Volume tulis tinggi; sifatnya riwayat (hampir append-only) sehingga konflik jarang; replikasi sinkron akan menghambat loket di jam sibuk.

Tabel	Jenis replikasi	Justifikasi sesuai kasus
	dan cadangan (tidak direplikasi penuh ke semua simpul)	

Kebutuhan atomik antara Pembayaran dan Status_Permohonan tidak diselesaikan dengan replikasi, melainkan dengan transaksi terdistribusi 2PC (lihat B4).

B3. (Sub-CPMK-1.3) Biaya Transmisi Join Warga (Simpul A) dengan Layanan_Surat (Simpul B)

Simbol (sesuai soal)

- n : jumlah baris tabel Warga di simpul A
- n_k : jumlah nilai kunci unik yang akan dikirim
- s_k : ukuran setiap nilai kunci (byte)
- m : jumlah baris tabel Layanan_Surat di simpul B
- s_L : ukuran setiap baris Layanan_Surat (byte)
- sel : proporsi baris Layanan_Surat yang cocok ($0 \leq sel \leq 1$)

Biaya transmisi diukur dalam jumlah byte yang dikirim antarsimpul. Biaya pemrosesan lokal diabaikan, dan hasil join diproses di simpul A (kantor pusat).

Strategi 1: mengirim seluruh Layanan_Surat ke simpul A

$C_1 = m \times s_L$

Strategi 2: semi-join

1. A mengirim n_k nilai kunci unik ke B. Biayanya $n_k \times s_k$.
2. B menyaring Layanan_Surat dan mengirim kembali hanya baris yang cocok ke A. Biayanya $sel \times m \times s_L$.
3. A menyelesaikan join secara lokal.

$C_2 = n_k \cdot s_k + sel \cdot m \cdot s_L$

Kondisi semi-join lebih hemat

$C_2 < C_1 \iff n_k \cdot s_k + sel \cdot m \cdot s_L < m \cdot s_L$

$\iff n_k \cdot s_k < (1 - sel) \cdot m \cdot s_L$

$\iff sel < 1 - (n_k \cdot s_k) / (m \cdot s_L)$

Semi-join lebih hemat bila proporsi baris yang cocok kecil dan biaya mengirim kunci ($n_k \cdot s_k$) jauh lebih kecil daripada biaya baris Layanan_Surat yang berhasil disaring ($(1 - sel) \cdot m \cdot s_L$). Jika hampir semua baris cocok (sel ≈ 1), mengirim seluruh tabel lebih hemat.

Contoh angka: $n_k = 5.000$; $s_k = 16$ B (NIK 16 digit); $m = 200.000$; $s_L = 150$ B; $sel = 0,2$.

- $C_1 = 200.000 \times 150 = 30.000.000$ B
- $C_2 = 5.000 \times 16 + 0,2 \times 200.000 \times 150 = 80.000 + 6.000.000 = 6.080.000$ B
- Ambang: $sel < 1 - 80.000/30.000.000 \approx 0,997$. Karena $0,2 < 0,997$, semi-join lebih hemat (turun ± 80%).

B4. (Sub-CPMK-2.1 dan 2.2) Koordinator (A) Gagal setelah Fase Persiapan, Jaringan ke B Terputus

Langkah 2PC yang seharusnya berjalan

1. Fase 1 selesai: A (koordinator) sudah mengirim PREPARE dan B (peserta, Status_Permohonan) sudah menulis <READY T> lalu memberi YES. Pembayaran di A juga siap.
2. Fase 2 terganggu: A gagal sebelum mengirim keputusan akhir.
3. Pemulihan koordinator: A membaca log setelah pulih. Jika ada <COMMIT T>, A mengirim ulang GLOBAL-COMMIT sampai B memberi ACK. Jika tidak ada catatan keputusan (hanya PREPARE), A memutuskan ABORT (*presumed abort*) dan mengirim GLOBAL-ABORT.
4. Peserta B: berstatus *in-doubt*. B menunggu, menahan kunci, dan berulang kali menanyakan keputusan ke A sampai koneksi pulih. B baru melepas kunci setelah menerima keputusan, lalu mengirim ACK.

Evaluasi konsistensi

- Transaksi tidak boleh disimpulkan berhasil maupun gagal; statusnya *menunggu keputusan*.
- Keamanan konsistensi (*safety*) tetap terjaga: tidak ada data setengah-setengah secara permanen, karena kedua simpul akhirnya commit atau abort bersama.
- Dampak pada ketersediaan (*liveness*): baris Pembayaran dan Status_Permohonan milik warga tersebut terkunci sehingga transaksi lain pada data itu tertahan. Jika A sempat commit lokal, untuk sementara terlihat ketidaksesuaian (pembayaran tercatat, status belum berubah) sampai B menerima keputusan.

Apakah peserta boleh memutuskan commit atau abort sendiri?

Tidak boleh. Setelah memberi vote YES, B telah menyerahkan hak memutuskan kepada koordinator. Jika B meng-commit sendiri padahal keputusan A adalah abort (atau sebaliknya), atomicity rusak dan data A dan B bertentangan. Satu-satunya jalan aman adalah menunggu keputusan koordinator. Pada kasus ini hanya ada dua simpul, sehingga B tidak dapat bertanya ke peserta lain (*cooperative termination protocol* baru berguna bila ada peserta ketiga).

Mitigasi dampak penguncian data

1. Log koordinator yang durable dan koordinator cadangan (*standby*) yang dapat membaca log dan melanjutkan keputusan, sehingga masa tunggu singkat.
2. Timeout dan pengiriman ulang otomatis keputusan serta pertanyaan status.
3. Mengurangi cakupan dan durasi kunci: *row-level locking*, transaksi pendek, dan *lock timeout*.
4. Pemantauan transaksi in-doubt dengan peringatan, dan penyelesaian manual oleh administrator sebagai pilihan terakhir (dicatat dan direkonsiliasi, karena berisiko *heuristic hazard*).
5. Jaringan redundan antara simpul A dan B.
6. Komunikasi status ke pengguna: tampilkan status “sedang diproses” agar warga tidak mengulang pembayaran.

7. Alternatif arsitektur: protokol berbasis konsensus (Paxos Commit) atau pola *saga* dengan transaksi kompensasi bila pemblokiran tidak dapat diterima. Three-phase commit mengurangi pemblokiran tetapi tidak aman terhadap partisi jaringan.